

LUCRAREA NR. 5

Fișiere de comenzi - noțiuni de bază

Lucrarea prezintă modul de funcționare al fișierelor executabile în *Linux* precum și metodele de realizare și modul de funcționare al fișierelor de comenzi (*shell scripts*).

5.1. Fișiere executabile

În sistemul de operare *Linux* există două tipuri de fișiere executabile:

- fișiere executabile de tip *ELF* (*Executable Linking Format*);
- fișiere executabile de tip *script* (fișier de comenzi sau *script shell*).

Fișierele executabile de tip *ELF* sunt fișiere compilate binar (conținut criptat de compilator) iar fișierele de comenzi sunt fișiere text care conțin seturi de comenzi specifice interpretorului de comenzi utilizat.

Primele caractere ale unui fișier executabil de tip *ELF* sunt chiar caracterele *ELF*, restul caracterelor reprezentând date criptate de compilator. În cazul fișierelor de comenzi, prima linie este întotdeauna numele interpretorului de comenzi, precedat de caracterele “#!” (ex. “#!/bin/bash”).

Un fișier executabil se poate lansa în execuție doar dacă are setate drepturile de execuție. Odată setate aceste drepturi, fișierul executabil se apelează astfel: *.nume_fișier* (figura 5.1)

```
root@slax:~$ cat > exec.sh
#!/bin/bash
echo "Exemplu fisier executabil"
root@slax:~$ ls -al exec.sh
-rw-r--r-- 1 root root 45 2008-10-29 13:40 exec.sh
root@slax:~$ chmod +x exec.sh
root@slax:~$ ls -al exec.sh
-rwxr-xr-x 1 root root 45 2008-10-29 13:40 exec.sh
root@slax:~$ ./exec.sh
Exemplu fisier executabil
root@slax:~$ █
```

Figura 5.1. Apelarea unui fișier executabil.

Fișierele executabile care se află în directoarele `/bin`, `/sbin`, `/usr/bin` și `/usr/sbin` nu trebuie să fie precedate de `./` în momentul apelării. Interpretorul de comenzi va căuta numele executabilului apelat de la tastatură în aceste directoare. Dacă executabilul nu este găsit, interpretorul va afișa mesajul de eroare: `-bash: comanda: command not found`.

5.2. Variabile

Variabilele sunt seturi de date care conțin diferite informații utilizabile de către aplicații sau de către alte variabile. În interpretoarele de comenzi din *Linux* există trei tipuri de variabile: variabile sistem, variabile ale scripturilor shell și variabile definite de utilizatori. Orice variabilă în *Linux* se notează cu `"$nume_variabilă"`. În tabelul 5.1 sunt prezentate o parte din variabilele sistem împreună cu semnificația lor.

Tabelul 5.1. Variabile sistem în Linux

Variabilă	Semnificație
\$HOME	conține numele directorului personal al utilizatorului curent
\$HOSTNAME	conține numele sistemului în rețea
\$MACHTYPE	conține numele arhitecturii sistemului
\$OSTYPE	conține numele sistemului de operare
\$PATH	conține lista de directoare în care <i>shell</i> -ul va căuta executabilele
\$PWD	conține numele directorului curent
\$SHELL	conține numele interpretorului de comenzi utilizat
\$TERM	conține numele terminalului (consolei)
\$UID	conține ID-ul utilizatorului curent
\$USER	conține numele contului utilizatorului curent

Variabilele pot fi definite în două moduri: `nume_variabilă="valoare"` sau `export nume_variabilă="valoare"`.

Valoarea unei variabile se poate afișa cu ajutorul comenzii *echo* (figura 5.2).

echo *opțiuni* `$nume_variabilă` - afișează valoarea unei variabile.

Opțiuni:

-n - păstrează cursorul în linia curentă după afișare

```
root@slax:~$ export var1="Continut variabila var1"
root@slax:~$ echo $var1
Continut variabila var1
root@slax:~$ echo -n $var1
Continut variabila var1root@slax:~$
```

Figura 5.2. Comanda echo.

5.3. Scripturi Shell

Un *script shell* este un fișier executabil care conține seturi de comenzi de bază și/sau comenzi specifice *shell*-urilor. Orice *script shell* este inițial un fișier text în care se scriu comenzile necesare. După editare, scripturile trebuie transformate în fișiere executabile utilizând comanda "*chmod*". Lansarea în execuție a unui *script shell* se poate face în trei moduri:

- ***./nume_fișier* argumente** - dacă fișierul executabil se află în directorul curent;
- ***/locăție/./nume_fișier* argumente** - dacă fișierul nu se află în directorul curent;
- ***nume_fișier* argumente** - dacă fișierul executabil se află într-unul din directoarele salvate în variabila \$PATH.

Parametrul "*argumente*" reprezintă valorile care se transferă spre *scriptul shell* direct din linia de comandă. Dacă se dorește transferul mai multor valori, acestea vor fi delimitate prin *spațiu* iar dacă valorile conțin *spațiu* acestea vor fi încadrate între ghilimele (ex. *./script 12 "ab cd"* - unde "12" reprezintă valoarea argumentului 1 iar "ab cd" valoarea argumentului 2).

Într-un *script shell* comenzile pot fi scrise pe linii separate sau în aceeași linie delimitate de ";". Prima linie a unui *script* va reprezenta întotdeauna locația și numele interpretorului de comenzi (ex. *#!/bin/bash*).

Exemplul 1 - Să se scrie un *script shell* care să afișeze numele directorului curent.

```
#!/bin/bash
echo "Directorul curent este: $PWD"
```

În cadrul unui *script shell* variabilele se pot defini/modifica la fel ca și în linia de comandă (tabelul 5.2).

Tabelul 5.2. Variabile interne - atribuirea valorilor

Tip atribuire	Explicații
var1="exemplu text"	atribuie variabilei <i>var1</i> valoarea <i>exemplu text</i>
var1=1116	atribuie variabilei <i>var1</i> valoarea întreagă 1116
var1=\$PWD	atribuie variabilei <i>var1</i> valoarea altei variabile (\$PWD)
read var1	citește de la tastatură valoarea variabilei <i>var1</i>

Exemplul 2 - Să se scrie un *script shell* care să citească de la tastatură valoarea variabilei *var1*, să atribuie această valoare variabilei *var2* și să afișeze valoarea variabilei *var2*.

```
#!/bin/bash
echo -n "Introduceți valoarea variabilei var1:"
read var1
var2=$var1
echo "Valoarea variabilei var2 este: $var2"
```

Pe lângă variabilele definite de utilizator, *scripturile shell* mai conțin și un set de variabile predefinite. Valorile acestor variabile pot fi utilizate în interiorul *scripturilor* ca și cele definite de utilizator, cu diferența că, pentru o parte din ele, atribuirea se face automat de către *script* în momentul lansării în execuție (tabelul 5.3).

Tabelul 5.3. Variabile predefinite în scripturile shell

Variabilă	Semnificație
\$#	numărul de argumente cu care se lansează în execuție <i>scriptul</i>
\$*	valoarea concatenată a argumentelor
\$@	valorile tuturor argumentelor sub formă de listă
\$0	numele <i>scriptului</i>
\$N	valoarea argumentului <i>N</i> , unde <i>N</i> reprezintă numărul de ordine al argumentului (ex. <i>./script 12</i> - \$1 = 12)
\$\$	ID-ul procesului generat de <i>script</i>
\$(comandă)	valoarea returnată de <i>comandă</i> (ex. <i>\$(ls)</i> - valoarea returnată de comanda <i>ls</i>)
Continuare în pagina următoare	

Tabela 5.3 – continuare

Variabilă	Semnificație
<code>\$((a+b))</code>	valoarea rezultată în urma unei operații matematice (ex. <code>var1=\$((a+b))</code> - unde <code>a</code> și <code>b</code> sunt variabile)
<code>\${variabilă}text</code>	adaugă un text la valoarea unei variabile

Exemplul 3 - Să se scrie un *script shell* care să afișeze numele *scriptului*, numărul de argumente și valoarea primelor două argumente.

```
#!/bin/bash
echo "Numele scriptului este: $0"
echo "Numarul de argumente este: $#"
```

```
echo "Valoarea argumentului 1 este: $1"
echo "Valoarea argumentului 2 este: $2"
```

Într-un *script shell* operațiile matematice se pot realiza în trei moduri:

- `let "variabilă=expresie"`
- `variabilă='expr expresie'`
- `variabilă=$(( expresie ))`

Exemplul 4 - Să se scrie un *script shell* care să calculeze suma a două numere citite de la tastatură.

```
#!/bin/bash
echo -n "Introduceti valoarea lui a: "
```

```
read a
echo -n "Introduceti valoarea lui b: "
```

```
read b
let "c = a + b"
echo "Suma numerelor este: $c"
```

Exemplul 5 - Să se scrie un *script shell* care să calculeze suma și produsul a două numere introduse ca argumente la numele *scriptului*.

```
#!/bin/bash
a=$1
b=$2
c=`expr $a + $b`
d=$((a * b))
echo "Suma numerelor este: $c"
echo "Produsul numerelor este: $d"
```

5.4. Funcții definite de utilizator

Utilizarea funcțiilor poate simplifica în mare măsură un program. Ca și în limbajele de programare, în *scripturile shell* utilizatorul poate defini propriile funcții. Definirea unor funcții proprii este utilă în cazul în care același set de comenzi trebuie folosit de mai multe ori într-un program.

Sintaxa unei funcții în *scripturile shell* este următoarea:

```
function nume_functie {
    comanda1
    comanda2
    ...
    comandaN
}
```

Aplearea funcțiilor definite de utilizator se realizează prin "*nume_functie argumente*". În cadrul funcțiilor, variabilele predefinite (tabelul 5.3) preiau valorile argumentelor cu care se apelează funcțiile. De exemplu, dacă se definește funcția "*suma*" și se apelează în *script* cu "*suma 12 14*", variabila "\$1", în corpul funcției, va avea valoarea 12.

Exemplul 6 - Să se scrie un *script shell* care să calculeze suma a două numere cu ajutorul unei funcții definite în interiorul *scriptului*.

```
#!/bin/bash
function suma {
    a=$1
    b=$2
    c=$(( $a + $b ))
    echo "Suma este: $c"
}
echo -n "a= "
read a
echo -n "b= "
read b
suma $a $b
```

5.5. Exerciții propuse

- 1) Să se scrie un *script shell* care să calculeze suma și produsul a 4 numere întregi introduse de la tastatură.
- 2) Să se scrie un *script shell* care să calculeze suma și produsul a 3 numere întregi introduse ca argumente la numele *scriptului*, utilizând funcții definite de utilizator.
- 3) Să se găsească și să se corecteze erorile din următorul *script shell*:

```
#!/bin/bash
function produs {
    nr1=$1
    nr2=$2
    c=$(( $nr1 * $nr2 ))
    echo "Produsul este: c"
}
echo -n "a= "
read $a
echo -n "b= "
read $b
produs a b
```

LUCRAREA NR. 6

Fișiere de comenzi - instrucțiuni de control

Scopul acestei lucrări este înțelegerea modului de funcționare instrucțiunilor de control decizionale (*if*, *else*, *case*) și a celor de tip *buclă* (*for*, *while*, *until*) în cadrul fișierelor de comenzi.

6.1. Instrucțiuni decizionale

Scripturile shell funcționează, într-o anumită măsură, ca și limbajele de programare. Acestea permit folosirea instrucțiunilor decizionale (*if*, *else*, *case*) pentru compararea valorilor anumitor variabile, indiferent de tipul de date conținut de acestea.

Spre deosebire de limbajele de programare, în scripturile shell, operatorii folosiți pentru compararea datelor sunt diferiți, în funcție de tipul de date.

În tabelul 6.1 sunt prezentați operatorii care pot fi utilizați pentru compararea numerelor întregi.

Tabelul 6.1. Operatori de comparare pentru numere întregi

Operator	Semnificație
\$var1 -eq \$var2	returnează TRUE dacă var1 este egală cu var2
\$var1 -ge \$var2	returnează TRUE dacă var1 este mai mare sau egală ca var2
\$var1 -gt \$var2	returnează TRUE dacă var1 este mai mare ca var2
\$var1 -le \$var2	returnează TRUE dacă var1 este mai mică sau egală ca var2
\$var1 -lt \$var2	returnează TRUE dacă var1 este mai mică decât var2
\$var1 -ne \$var2	returnează TRUE dacă var1 nu este egală cu var2

În tabelul 6.2 sunt prezentați operatorii care pot fi utilizați pentru compararea șirurilor de caractere.

Tabelul 6.2. Operatori de comparare pentru șiruri de caractere

Operator	Semnificație
\$var1 = \$var2	returnează TRUE dacă șirurile var1 și var2 sunt identice
Continuare în pagina următoare	

Tabela 6.2 – continuare

Operator	Semnificație
\$var1 != \$var2	returnează TRUE dacă șirurile var1 și var2 nu sunt identice
-n \$var1	returnează TRUE dacă lungimea șirului var1 nu este zero
-z \$var1	returnează TRUE dacă lungimea șirului var1 este zero

În tabelul 6.3 sunt prezentați operatorii care pot fi utilizați pentru compararea fișierelor și directoarelor.

Tabelul 6.3. Operatori de comparare pentru fișiere și directoare

Operator	Semnificație
-f \$var1	returnează TRUE dacă var1 este fișier
-d \$var1	returnează TRUE dacă var1 este director
-r \$var1	returnează TRUE dacă var1 este un fișier sau director cu drept de citire
-w \$var1	returnează TRUE dacă var1 este un fișier sau director cu drept de scriere
-x \$var1	returnează TRUE dacă var1 este un fișier sau director cu drept de execuție

Blocurile logice *if* pot fi folosite în trei moduri:

1) *if-then*

Sintaxă:

```
if [ conditie ]
then
    comanda1
    comanda2
    ...
    comandaN
fi
```

2) *if-then-else*

Sintaxă:

```
if [ conditie ]
then
    comanda1
    comanda2
    ...
    comandaN
else
    comanda1
    comanda2
    ...
    comandaN
fi
```

3) *if-then-elif-then-else*

Sintaxă:

```
if [ conditie ]
then
    comanda1
    comanda2
    ...
    comandaN
elif [ conditie ]
then
    comanda1
    comanda2
    ...
    comandaN
else
    comanda1
    comanda2
    ...
    comandaN
fi
```

Exemplul 1 - Să se scrie un script shell care să citească o valoare întreagă de la tastatură și să verifice dacă valoarea variabilei este egală, mai mică sau mai mare decât zero.

```
#!/bin/bash
echo -n "Introduceți valoarea variabilei var1:"
read var1
if [ $var1 -eq 0 ]
then
    echo "var1 este egala cu 0"
elif [ $var1 -lt 0 ]
then
    echo "var1 este mai mica decat 0"
else
    echo "var1 este mai mare ca 0"
fi
```

Instrucțiunea **case** compară valoarea unei variabile cu valorile existente într-o listă predefinită și execută setul de comenzi corespunzător valorii găsite (dacă există).

Sintaxa instrucțiunii **case** este următoarea:

```
case $nume_variabila in
    valoare1)
        comanda1;
        comanda2;
        ...
        comandaN;;
    valoare2)
        comanda1;
        comanda2;
        ...
        comandaN;;
    *)
        comanda1;
```

```

        comanda2;
        ...
        comandaN;;
    esac

```

Exemplul 2 - Să se scrie un script shell care să identifice valoarea unei variabile într-o listă de valori.

```

#!/bin/bash
echo -n "Introduceti valoarea variabilei var1:"
read var1
case $var1 in
    1)
        echo "var1=1";;
    2)
        echo "var1=2";;
    3)
        echo "var1=3";;
    *)
        echo "var1 nu este in lista de valori";;
esac

```

Caracterul "*" reprezintă valoarea *default*, adică setul de comenzi care trebuie executat dacă variabila are orice valoare care nu există în lista de valori. Pentru orice set de comenzi se pot scrie mai multe valori, delimitate prin caracterul "|":

```

case $nume_variabila in
    val1|val2|val3)
        comenzi;
        ...
esac

```

6.2. Instrucțiuni de tip buclă

Instrucțiunile de tip buclă pot fi folosite pentru parcurgerea unor liste de la valori (instrucțiunea *for*) sau pentru executarea condiționată a unor seturi de

Fișiere de comenzi - instrucțiuni de control

comenzi (instrucțiunile *while* și *until*).

Instrucțiunea **for** parcurge o listă de valori atribuind, pe rând, fiecare valoare unei variabile.

Sintaxă:

```
for var1 in sir_de_valori
do
    comanda1
    comanda2
    ...
    comandaN
done
```

Parametrul *sir_de_valori* poate fi un șir de numere întregi, un șir de caractere și/sau cuvinte sau o variabilă care conține un șir de valori. Condiția ca șirul să fie valid este să conțină valori delimitate prin spațiu.

Exemplul 3 - Să se scrie un script shell care să afișeze numărul de fișiere și de directoare din directorul curent.

```
#!/bin/bash
lista=$( ls )
fisiere=0
directoare=0
for nume in $lista
do
    if [ -f $nume ]
    then
        let "fisiere=fisiere+1"
    fi
    if [ -d $nume ]
    then
        let "directoare=directoare+1"
    fi
done
echo "Numarul de fisiere: $fisiere"
echo "Numarul de directoare: $directoare"
```

Instrucțiunea *while* execută un set de comenzi atâta timp cât condiția de execuție este adevărată.

Sintaxă:

```
while [ conditie ]
do
    comanda1
    comanda2
    ...
    comandaN
done
```

Exemplul 4 - Să se scrie un script shell care să creeze N fișiere având numele de forma *fișier.1*, *fișier.2*, ..., *fișier.N*.

```
#!/bin/bash
echo -n "Introduceti numele fisierului"
read nume
echo -m "Introduceti numarul de fisiere"
read numar
i=0
while [ $i -le $numar ]
do
    fisier=${nume}.$i
    touch $fisier
    let "i=i+1"
done
echo "Fisierele au fost create"
ls ${nume}.*
```

Instrucțiunea *until* execută un set de comenzi până când condiția de execuție devine adevărată.

Sintaxă:

```
until [ conditie ]
do
```

Fișiere de comenzi - instrucțiuni de control

```
comanda1
comanda2
...
comandaN
done
```

Exemplul 5 - Să se scrie un script shell care să numere fișierele și subdirectoarele din diferite directoare citite de la tastatură până când numele directorului introdus este *"exit"*. Să se afișeze numărul de directoare parcurse, numărul de fișiere găsite și numărul de subdirectoare găsite.

```
#!/bin/bash
echo -n "Introduceti numele unui director: "
read var
lista=$(ls $var)
fisiere=0
directoare=0
i=1
until [ $var = "exit" ]
do
    for nume in $lista
    do
        locatie=${var}/${nume}
        echo "Nume complet: $locatie"
        if [ -f $locatie ]
        then
            let "fisiere=fisiere+1"
        fi
        if [ -d $locatie ]
        then
            let "directoare=directoare+1"
        fi
    done
    echo -n "Introduceti alt director [$i]: "
    read var
```

```

if [ $var != "exit" ]
then
    let "i=i+1"
    lista=$(ls $var)
fi
done
echo "Numarul de directoare parcurse: $i"
echo "Numarul de fisiere gasite: $fisiere"
echo "Numarul de directoare gasite: $directoare"

```

6.3. Exerciții propuse

- 1) Să se scrie un script shell care să calculeze suma a N numere întregi introduse ca argumente la numele scriptului.
- 2) Să se scrie un script shell care să afișeze numărul minim și maxim dintr-un șir de numere întregi introduse ca argumente la numele scriptului.
- 3) Să se scrie un script shell care să afișeze timpul în care unei variabile i se atribuie automat valori de la 0 la 200000 (vezi comanda *date +%s*).
- 4) Să se scrie un script shell care să calculeze suma, diferența sau produsul a două numere în funcție de operatorul introdus de la tastatura (+, - sau *).
- 5) Să se găsească și să se corecteze erorile din următorul script shell:

```

#!/bin/bash
function produs (
    prod=1
    lista=$@
    for $val in lista
    do
        if [ val -ne 0 ]
        then
            let prod=prod*val
        fi
    done
}

```


Fişiere de comenzi - instrucţiuni de control

```
done
    echo "Produsul numerelor este: prod"
)
arg=$@
produs $arg
```